# MediScript

## Developer & Data Guide

The deeper companion to the Editing Guide: how every part fits together, what each folder and file does, how the prescription pipeline works, how to grow the medicine datasets, and concrete ideas for extending the app. Written to stay easy to follow.

Contents: 1) How it works end-to-end 2) Full file map 3) The reading pipeline 4) The search pipeline 5) Data files explained 6) Extending the datasets 7) Services reference 8) API reference 9) Database 10) Config knobs 11) Add a new page 12) Add a new feature 13) Future ideas & roadmap 14) Testing & deploy 15) Troubleshooting

## 1 · How it works, end to end

MediScript is a **FastAPI** web app (Python) that serves HTML pages and small JSON 'endpoints'. A browser loads a page; the page's JavaScript calls an endpoint; the endpoint runs a 'service' and returns data; the page shows it. In order:

| # | What happens | Where in the code |
|---|---|---|
| 1 | Browser opens a page (e.g. /app/upload) | app/api/pages.py -> app/templates/... |
| 2 | Page JS calls an endpoint (e.g. /api/user/prepare-image) | app/api/user_api.py |
| 3 | Endpoint calls a service to do the real work | app/services/*.py |
| 4 | Service reads data / calls OCR / matches medicines | app/services/ + data/ |
| 5 | Result saved to the database (if needed) | app/models/ via SQLAlchemy |
| 6 | JSON returned; the page renders it for the user | the template script block |

**NOTE**

Two 'portals' share this flow: **User** (patients) and **Chemist** (pharmacies). They have separate pages and endpoints but reuse the same services (OCR, matching, prices).

## 2 · Full file map (every folder, plain English)

| Path | What it does |
|---|---|
| app/main.py | Starts the app; wires all endpoints, templates and static files together. |
| app/core/config.py | All settings & tuning knobs (keys, thresholds, model on/off). |
| app/core/database.py | Connects to the database; creates tables on startup. |
| app/core/security.py | Password hashing and login tokens (JWT). |
| app/core/deps.py | 'Who is logged in?' checks used to protect pages. |
| app/api/pages.py | Routes that return HTML pages (the visible URLs). |

| Path | What it does |
|---|---|
| app/api/auth.py | Register, login, logout. |
| app/api/account_api.py | View/update profile, change password. |
| app/api/user_api.py | Patient actions: upload, read region, search, medicine info, votes. |
| app/api/chemist_api.py | Pharmacy actions: dispense, inventory CRUD, demand, recalls. |
| app/api/guest_api.py | The no-signin demo (1 free upload + 1 search). |
| app/services/ | The 'brains' — one file per job (see section 7). |
| app/models/ | Database tables: user, prescription, inventory, demand, analytics, guest. |
| app/schemas/__init__.py | Shapes of incoming JSON (e.g. a drawn region's coordinates). |
| app/templates/ | All web pages (HTML). base.html is the shared frame. |
| app/static/css/style.css | Every colour, font and layout rule. |
| app/static/js/app.js | Shared browser helpers (fetch wrappers, form handling). |
| data/ | Medicine datasets + your drop-in folder (see sections 5-6). |
| scripts/ | Helpers: build the DB, deploy to GitHub, enable the AI model, seed a demo. |
| migrations/ | Database version history (for Postgres upgrades). Ignore for SQLite. |
| tests/ | Automated checks. Run with python -m pytest. |
| render.yaml / DEPLOY.md | Free-tier deployment blueprint + step-by-step. |
| .env | Your private settings (OCR key). You create it; it's kept out of GitHub. |

## 3 · The prescription-reading pipeline

This is the heart of the app. When a prescription photo is read, it flows through these steps (all orchestrated by `app/services/extraction.py`):

| Step | What it does | File |
|---|---|---|
| 1. OCR | Photo -> text, using OCR.space Engine 3 (fixed by design). | ocr_service.py |
| 2. Detect | Decide which lines are actually medicines (not letterhead / vitals). | drug_ner.py |
| 3. Match | Compare those lines to the medicine list; score each. | matcher.py + text_processing.py |
| 4. Judge | Label each as confident / unsure / can't-read (honest states). | extraction.py (classify_region) |
| 5. Enrich | On demand: composition, price, savings, uses/side-effects. | medicine_details.py + medicine_info.py |

**NOTE** Step 2 is why the hospital name no longer matches a medicine. It looks for drug 'signals' (a dose like 500 mg, a frequency like 1-0-1, a form like Tab/Inj) and a stopword list for headers/vitals. Optionally a real AI model (Med7) can add a signal — see section 13.

**RULE** Never change the OCR request in ocr_service.py (Engine 3 is a fixed requirement). Improve the steps AFTER it instead.

## 4 · The search pipeline

Typing a medicine name flows through:

| Step | What / where |
|---|---|
| Rank names | search.py ranks the ~199k names by how well they match what you typed. |
| Show details | medicine_info.py assembles composition (local) + uses/side-effects (openFDA on the salt). |
| Price & savings | medicine_details.py adds the price and the cheapest same-formula substitute. |
| Verify name | name_resolver.py confirms via RxNorm only on an EXACT match (never a guess). |
| Buy links | pharmacy_links.py builds 'search on 1mg / PharmEasy / Netmeds / Apollo' links. |

## 5 · The data files explained

Everything lives in `data/`. Three files are generated and used at runtime; the rest are raw ingredients.

| File | What it is | Used for |
|---|---|---|
| medicines_master.csv | ~6,500 accurate generic + curated names. | Matching prescriptions (the safe list). |
| medicines_search.csv | ~199,000 names incl. Indian brands. | The search box / autocomplete. |
| brand_details.csv.gz | ~200,000 brands with price + composition + salts. | Composition, substitutes, savings. |
| sources/ | Raw inputs: Open BNF (UK), curated India, Indian brands & generics. | Rebuilding the 3 files above. |
| datasets/ | YOUR drop-in folder — add CSV/TXT here. | Auto-merged into search on rebuild. |
| medicines_seed.csv / demo/ | Small samples for tests / demo. | Seeding a demo pharmacy. |

**NOTE**

The three generated files SHIP in the repo, so the app runs immediately with no build step. You only rebuild when you add or update data.

## 6 · Extending the medicine datasets

**The easy path (no code):**

| Step | Action |
|---|---|
| 1 | Get a real dataset (see sources below). Download the CSV or make a TXT. |
| 2 | Drop the file into data/datasets/ (as many files as you like). |
| 3 | Run python scripts/build_medicine_db.py |
| 4 | Restart the app. New names are searchable immediately. |

**Accepted formats** (auto-detected):

```
# CSV: any column whose header contains one of:
#   name, medicine, drug, brand, product, composition
name,manufacturer
Dolo 650 Tablet,Micro Labs
Crocin Advance,GSK

# TXT: one medicine name per line
Paracetamol
Amoxicillin
```

**What the build script does** (scripts/build_medicine_db.py): reads the bundled sources + anything in data/datasets/ + any files you pass with --extra, then writes the three runtime files. Brand names are auto-cleaned (dosage/pack/form

suffixes stripped) so they match well. Everything is de-duplicated.

```
# add one-off files without moving them:
python scripts/build_medicine_db.py --extra data/sources/mine.csv another.csv
```

**To also get composition, price & substitutes** for new brands, the dataset needs price + composition columns (like the Indian dataset). The script rebuilds brand_details.csv.gz from data/sources/indian_medicine_data.csv; point that at a richer CSV (same columns: name, price, short_composition1/2) to extend savings coverage.
**Good, real sources** (never fabricate medical data):

| Source | Gives you |
|---|---|
| Kaggle: A-Z Medicines Dataset of India | 250k+ brands + compositions + prices (already used). |
| Kaggle: Extensive A-Z Medicines Dataset | Adds uses / side-effects text you could store locally. |
| WHO Model List of Essential Medicines | Authoritative generic names. |
| Open BNF (bundled) | UK generic names. |
| Your national drug register | Country-specific brands (add via data/datasets/). |

**CAUTION** Files you drop in data/datasets/ are git-ignored by default (they can be large). Commit them explicitly if you want them in your repo / on Render.

**IDEA** Want another country? Just drop that country's brand CSV into data/datasets/ and rebuild — the search index grows automatically. For composition/price on those brands, add the columns and extend the brand_details build step.

## 7 · Services reference (the app/services/ folder)

| File | One-line job |
|---|---|
| ocr_service.py | Talks to OCR.space Engine 3; returns text + per-line boxes. (Fixed engine.) |
| drug_ner.py | Decides which lines are medicines; optional Med7/scispaCy model hook. |
| text_processing.py | Cleans text, makes candidate phrases and OCR-typo variants. |
| matcher.py | Fuzzy-matches text to medicine names; rejects fragment matches. |
| extraction.py | Runs the whole reading pipeline; saves prescriptions + detections. |
| medicine_db.py | Loads the two medicine name lists into memory (cached). |
| medicine_details.py | Composition, price, substitutes, and savings from the dataset. |
| medicine_info.py | Combines local composition with openFDA uses/side-effects + savings. |
| name_resolver.py | RxNorm exact-match verification + approximate suggestions. |
| pharmacy_links.py | Builds 'buy at' search links for 1mg/PharmEasy/Netmeds/Apollo. |
| search.py | Ranks search results (exact > prefix > word > fuzzy). |
| news_service.py | Health news (WHO/MedlinePlus/CDC RSS) + openFDA drug recalls. |
| analytics_service.py | Computes user and pharmacy statistics. |
| inventory_service.py | Stock checks, stats, CSV import, restock, demand. |

## 8 · API reference (the URLs pages call)

| Portal / file | Main endpoints |
|---|---|
| auth.py | /api/auth/register · /login · /logout |
| account_api.py | /api/account/me · /profile · /password |
| user_api.py | /api/user/prepare-image · /manual-region · /upload · /search · /medicine-info · /confirm · /vote · /analytics · /health-news |
| chemist_api.py | /api/chemist/prepare-image · /manual-region · /upload · /inventory (GET/POST/PATCH/DELETE) · /inventory/import · /restock · /analytics · /industry-news |
| guest_api.py | /api/guest/status · /upload · /search |
| pages.py | The visible page URLs: / · /demo · /login · /app/* · /chemist/* |

**TIP** To add data to a page, add a function in the matching api file and call it from the page script with MS.get('/api/...'). MS.get/post/patch/del live in app/static/js/app.js.

## 9 · The database (app/models/)

Uses SQLAlchemy. SQLite by default (a single file); Postgres if you set DATABASE_URL. Tables are created automatically on startup.

| Model file | Stores |
|---|---|
| user.py | Accounts (email, password hash, role: user/chemist, pharmacy name). |
| prescription.py | Each scan (Prescription) and each detected medicine (Detection). |
| inventory.py | Chemist stock: name, qty, price, rack, expiry, supplier, reorder level. |
| demand.py | DemandEvent + RestockRequest (what patients need). |
| analytics.py | VoteEvent, SearchEvent, StockEvent (event logs for stats). |
| guest.py | GuestUsage (counts free demo actions per browser). |

**CAUTION** If you add or rename a field on a model and use Postgres, create a migration (alembic revision --autogenerate -m 'msg' then alembic upgrade head). SQLite in dev just recreates tables on a fresh file.

## 10 · Config knobs (app/core/config.py)

| Setting | Meaning |
|---|---|
| secret_key | Signs login tokens. Auto-generated on Render. |
| ocrspace_keys | Your OCR.space key(s), comma-separated. Set in .env. |
| match_threshold (72) | Min score to accept a medicine match. Higher = stricter. |
| manual_threshold (60) | Threshold for a hand-drawn region. |
| database_url | Where data is stored. Blank = SQLite file. |
| rxnorm_base / openfda_base | Government APIs for verification & labels. |
| http_cache_ttl | How long external results are cached (seconds). |
| drug_ner_model | Blank = built-in detector; set to a spaCy model to use AI NER. |
| drug_ner_labels | Which entity labels count as drugs (DRUG, CHEMICAL). |
| allowed_origins | Which websites may call the API (CORS). |

## 11 · Walkthrough: add a new page

| Step | Do |
|---|---|
| 1 | Create app/templates/user/mypage.html — copy an existing page and change the content block. |
| 2 | In app/api/pages.py, add a route that returns that template (copy an existing one). |
| 3 | Add a link to it in app/templates/user/_nav.html. |
| 4 | If it needs data, add an endpoint in user_api.py and call it from the page script. |

## 12 · Walkthrough: add a new feature (example)

Say you want a 'Save to my medicines' button. The pattern is always the same:

| Layer | What you add |
|---|---|
| Model | A table to store it (e.g. SavedMedicine) in app/models/. |
| API | Endpoints to add/list/remove in user_api.py. |
| Service | (Optional) logic in app/services/ if it's non-trivial. |
| Page | A button + list in the template, wired with MS.post / MS.get. |

## 13 · Future ideas & roadmap

Concrete, buildable extensions — with where each would live. All respect the project rules (OCR.space only; never fabricate medical facts).

| Idea | How / where | Effort |
|---|---|---|
| Drug-interaction checker | Combine a user's medicines; pull interactions from openFDA labels per salt. New service + user page. | Medium |
| My Medicine Cabinet + reminders | Save medicines to a personal list; refill reminders. New model + page. | Medium |
| Turn on real AI NER (Med7) | Already wired — run scripts/enable_med7.sh, set DRUG_NER_MODEL. Best on a paid host. | Easy |
| Export a clean medicine list PDF | Reuse a PDF library to print a patient's read result. New endpoint. | Easy |
| More countries / bigger data | Drop national brand CSVs into data/datasets/, rebuild. Extend brand_details for prices. | Easy |
| Local uses/side-effects dataset | Store a curated uses/side-effects CSV so info works offline (less openFDA reliance). | Medium |
| Search autocomplete | Add a type-ahead endpoint over medicines_search.csv. | Easy |
| Password reset / email verify | Add email sending + token flow in auth.py. | Medium |
| Admin dashboard | A protected page to see platform stats / manage users. | Medium |
| Multi-language OCR | OCR.space supports a language param — add a selector. Test carefully. | Medium |
| Rate limiting for production | Add a limiter (e.g. slowapi) on public endpoints. | Easy |
| Mobile / PWA polish | Add a manifest + service worker for install-to-home-screen. | Medium |

**IDEA** Start with the **drug-interaction checker** or **medicine cabinet** — both add clear user value, reuse data you already have, and follow the model->API->page pattern in section 12.

RULE

Whatever you add: never invent dosages, uses, or safety info. Pull from real sources (openFDA, RxNorm, curated datasets) and label anything uncertain, exactly as the app does today.

## 14 · Testing & deploying

```
python -m pytest         # run all automated checks
uvicorn app.main:app --reload   # run locally at :8000
```

Deploy on Render's free tier: push to GitHub, then New + -> Blueprint -> pick the repo -> set OCRSPACE_KEYS -> Apply. Full steps and free-tier notes are in DEPLOY.md.

## 15 · Troubleshooting

| Symptom | Likely fix |
|---|---|
| Page edit not showing | Refresh; for CSS/JS do a hard refresh (Ctrl+F5). |
| 'No module named app' | Run from the project root; use the virtual env; use python -m pytest. |
| OCR errors / 403 | Set a valid OCRSPACE_KEYS in .env; the demo key is rate-limited. |
| No uses/side-effects / no news | Those need internet (openFDA / RSS); they fill in when deployed/online. |
| First request very slow | Normal cold start — it loads the 200k medicine list once. |
| Draw box not working | Ensure you clicked-and-dragged on the image after it loaded. |

Keep this with the Editing Guide. Editing Guide = quick changes; this = the how-and-why plus growing the data and the app.